

Spring Framework & Spring Boot

Complete Interview Preparation Guide

200+ Professional Questions & Answers

Covering Spring Core, Boot, MVC, Data JPA, Security,
AOP, Cloud, Microservices, Batch, Testing & More

2026 Edition

Table of Contents

1. Spring Core Framework
2. Dependency Injection & IoC Container
3. Spring Bean Lifecycle & Scopes
4. Spring Boot Fundamentals
5. Spring Boot Auto-Configuration & Starters
6. Spring Boot Actuator & Monitoring
7. Spring MVC & REST APIs
8. Spring Data JPA & Hibernate
9. Spring Security
10. Spring AOP (Aspect-Oriented Programming)
11. Spring Transaction Management
12. Spring Cloud & Microservices
13. Spring Batch
14. Spring WebFlux & Reactive Programming
15. Spring Testing
16. Spring Profiles & Configuration
17. Spring Caching
18. Spring Messaging (JMS, Kafka, RabbitMQ)
19. Spring Design Patterns
20. Scenario-Based & Advanced Questions

1. Spring Core Framework

Q: What is the Spring Framework?

A: Spring is a lightweight, open-source Java application framework that provides comprehensive infrastructure support for developing enterprise Java applications. It is built on two core principles: **Inversion of Control (IoC)** and **Aspect-Oriented Programming (AOP)**. Spring simplifies Java development by handling low-level concerns like transaction management, data access, security, and more, allowing developers to focus on business logic.

Q: What are the main modules of the Spring Framework?

A: The Spring Framework is organized into the following key modules:

- **Spring Core Container:** Core, Beans, Context, SpEL
- **Data Access/Integration:** JDBC, ORM, OXM, JMS, Transactions
- **Web:** Web, Web-MVC, Web-Socket, Web-Flux
- **AOP & Instrumentation:** AOP, Aspects, Instrumentation
- **Messaging:** Spring Messaging
- **Test:** Spring Test module

Q: What are the advantages of using Spring Framework?

A: Key advantages include:

- **Lightweight:** Minimal overhead, POJO-based development
- **Loose Coupling:** Through Dependency Injection
- **Modular:** Use only what you need
- **Testable:** DI makes unit testing easy
- **Declarative Support:** Transactions, caching, security via annotations
- **Integration:** Excellent integration with Hibernate, JPA, Kafka, etc.
- **Mature Ecosystem:** Large community, extensive documentation

Q: What is the difference between Spring Framework and Spring Boot?

A:

Aspect	Spring Framework	Spring Boot
Configuration	Manual XML or Java-based config	Auto-configuration
Setup	Requires significant boilerplate	Minimal setup with starters
Server	External server required	Embedded server (Tomcat, Jetty)
Deployment	WAR file to server	Executable JAR with embedded server
Opinionated	No — full flexibility	Yes — sensible defaults
Production	Manual monitoring setup	Built-in Actuator endpoints

Q: What is a Spring Configuration File?

A: A Spring configuration file defines beans, their dependencies, and configuration metadata. It can be XML-based (`applicationContext.xml`), Java-based (`@Configuration` classes), or annotation-based. Modern applications prefer Java-based and annotation-based configuration over XML.

Q: What is `ApplicationContext` vs `BeanFactory` ?

A:

Feature	BeanFactory	ApplicationContext
Bean Loading	Lazy initialization	Eager initialization
Internationalization	Not supported	Supported
Event Publishing	Not supported	Supported
AOP Integration	Manual	Automatic
Annotation Support	Limited	Full support
Use Case	Lightweight / mobile apps	Enterprise applications

`ApplicationContext` is a superset of `BeanFactory` and is the preferred choice for all standard applications.

Q: What are the types of ApplicationContext in Spring?

A: Common implementations include:

- `ClassPathXmlApplicationContext` — loads from classpath
- `FileSystemXmlApplicationContext` — loads from file system
- `AnnotationConfigApplicationContext` — loads from Java @Configuration classes
- `WebApplicationContext` — for web applications
- `GenericWebApplicationContext` — for programmatic registration

2. Dependency Injection & IoC Container

Q: What is Inversion of Control (IoC)?

A: IoC is a design principle where the control of object creation and lifecycle management is transferred from the application code to the Spring container. Instead of objects creating their own dependencies, the container injects them. This promotes loose coupling, better testability, and separation of concerns. The Spring IoC container reads configuration metadata and creates, configures, and manages beans.

Q: What is Dependency Injection (DI)?

A: Dependency Injection is a specific implementation of IoC where dependencies are supplied to a class rather than the class creating them. Spring supports three types of DI:

- **Constructor Injection:** Dependencies passed via constructor parameters (recommended)
- **Setter Injection:** Dependencies set via setter methods
- **Field Injection:** Dependencies injected directly into fields using `@Autowired` (not recommended for production)

Q: Why is Constructor Injection preferred over Field Injection?

A: Constructor injection is preferred because:

- **Immutability:** Dependencies can be declared `final`
- **Required Dependencies:** Ensures all required dependencies are available at construction time
- **Testability:** Easy to pass mock dependencies in unit tests without reflection
- **No Hidden Dependencies:** Constructor signature clearly declares what is needed
- **Framework Independence:** Works without Spring annotations in test code

```
// Recommended - Constructor Injection
@Service
public class OrderService {
    private final PaymentService paymentService;
    private final InventoryService inventoryService;

    public OrderService(PaymentService paymentService,
                        InventoryService inventoryService) {
        this.paymentService = paymentService;
        this.inventoryService = inventoryService;
    }
}
```

Q: What is `@Autowired` and how does it work?

A: `@Autowired` tells Spring to automatically resolve and inject a matching bean. Spring searches the context by type first, then by qualifier/name. It can be applied on constructors, setters, or fields. Since Spring 4.3, `@Autowired` is optional on single-constructor classes — Spring will automatically use it for injection.

Q: What happens when multiple beans of the same type exist? How do you resolve ambiguity?

A: When Spring finds multiple candidates for injection, it throws `NoUniqueBeanDefinitionException`. You can resolve this using:

- `@Primary` — marks one bean as the default choice
- `@Qualifier("beanName")` — specifies exactly which bean to inject
- Match by variable name — if the field/parameter name matches a bean name
- `@Resource(name="beanName")` — JSR-250 annotation, injects by name

```
@Service
public class NotificationService {
    private final MessageSender sender;

    public NotificationService(@Qualifier("emailSender") MessageSender sender) {
        this.sender = sender;
    }
}
```

Q: What is the difference between `@Component`, `@Service`, `@Repository`, and `@Controller`?

A: All are specializations of `@Component` and register beans in the Spring context. The differences are semantic and functional:

Annotation	Layer	Special Behavior
<code>@Component</code>	Generic	Base stereotype, no extra behavior
<code>@Service</code>	Business Logic	Semantic marker for service layer
<code>@Repository</code>	Data Access	Enables exception translation (DB exceptions → Spring's <code>DataAccessException</code>)
<code>@Controller</code>	Web Layer	Handles HTTP requests, enables request mapping

Q: What is `@Lazy` annotation?

A: `@Lazy` defers bean initialization until it is first requested, rather than at application startup. Useful for beans that are expensive to create and may not always be needed. Can be applied at class level or injection point.

```
@Service
@Lazy
public class HeavyReportService {
    // Only created when first injected/used
}
```

Q: What is the difference between `@Inject`, `@Autowired`, and `@Resource` ?

A:

Feature	@Autowired	@Inject	@Resource
Standard	Spring-specific	JSR-330 (javax.inject)	JSR-250 (javax.annotation)
Matching	By type → qualifier	By type → qualifier	By name → type
Required	required=false supported	Use Optional<T>	Not applicable

3. Spring Bean Lifecycle & Scopes

Q: What is a Spring Bean?

A: A Spring Bean is any object that is instantiated, configured, and managed by the Spring IoC container. Beans are defined through annotations (`@Component` , `@Bean`), XML, or Java configuration. The container manages their complete lifecycle — from creation to destruction.

Q: Explain the complete Spring Bean Lifecycle.

A: The full lifecycle is:

- **1. Instantiation:** Container creates the bean instance
- **2. Populate Properties:** Dependencies are injected
- **3. BeanNameAware:** `setBeanName()` called if implemented
- **4. BeanFactoryAware:** `setBeanFactory()` called if implemented
- **5. ApplicationContextAware:** `setApplicationContext()` called
- **6. BeanPostProcessor (before):** `postProcessBeforeInitialization()`
- **7. @PostConstruct:** Custom initialization method
- **8. InitializingBean:** `afterPropertiesSet()`
- **9. Custom init-method:** Method specified in `@Bean(initMethod=...)`
- **10. BeanPostProcessor (after):** `postProcessAfterInitialization()`
- **11. Bean is Ready** — available for use
- **12. @PreDestroy:** Cleanup before destruction
- **13. DisposableBean:** `destroy()`
- **14. Custom destroy-method:** Method specified in `@Bean(destroyMethod=...)`

Q: What are the different Bean Scopes in Spring?

A:

Scope	Description	Context
<code>singleton</code>	One instance per Spring container (default)	All
<code>prototype</code>	New instance every time requested	All
<code>request</code>	One instance per HTTP request	Web only
<code>session</code>	One instance per HTTP session	Web only
<code>application</code>	One instance per ServletContext	Web only
<code>websocket</code>	One instance per WebSocket session	Web only

Q: What happens when you inject a Prototype bean into a Singleton bean?

A: The prototype bean is created only once (when the singleton is initialized) and the same prototype instance is reused for every call. This defeats the purpose of the prototype scope. Solutions include:

- `@Lookup` method injection — Spring overrides the method to return a new prototype each time
- `ObjectFactory<T>` or `Provider<T>` — call `.getObject()` each time
- `ApplicationContext.getBean()` — not recommended, couples to container
- Scoped proxy (`@Scope(proxyMode = ScopedProxyMode.TARGET_CLASS)`)

```
@Service
public class OrderService {
    @Lookup
    protected ShoppingCart createCart() {
        return null; // Spring overrides this
    }

    public void processOrder() {
        ShoppingCart cart = createCart(); // new instance each time
    }
}
```

Q: What is `@PostConstruct` and `@PreDestroy` ?

A: `@PostConstruct` marks a method to be called after dependency injection is complete — used for initialization logic. `@PreDestroy` marks a method to be called before the bean is removed from the container — used for cleanup. These are JSR-250 annotations and are the preferred way to handle lifecycle callbacks.

Q: What is a `BeanPostProcessor` ?

A: `BeanPostProcessor` is an interface that allows custom modification of bean instances before and after initialization. Spring uses it internally for `@Autowired`, `@Value`, and AOP proxying. You can create custom post-processors for cross-cutting initialization logic.

```
@Component
public class CustomBeanPostProcessor implements BeanPostProcessor {
    public Object postProcessBeforeInitialization(Object bean, String name) {
        // modify bean before @PostConstruct
        return bean;
    }
    public Object postProcessAfterInitialization(Object bean, String name) {
        // modify bean after initialization (AOP proxies created here)
        return bean;
    }
}
```

Q: What is a `BeanFactoryPostProcessor` ?

A: `BeanFactoryPostProcessor` allows modification of bean definitions (metadata) before any beans are created. It works at the configuration level.

`PropertySourcesPlaceholderConfigurer` is a well-known example — it resolves `$ {...}` placeholders. Key difference from `BeanPostProcessor`: it modifies bean definitions, not bean instances.

4. Spring Boot Fundamentals

Q: What is Spring Boot and why was it created?

A: Spring Boot is an opinionated framework built on top of Spring Framework that simplifies the creation of production-ready Spring applications. It was created to eliminate boilerplate configuration, provide embedded servers, and enable rapid application development. Key philosophy: **"Convention over Configuration"**.

Q: What are the key features of Spring Boot?

A: Core features include:

- **Auto-Configuration:** Automatically configures beans based on classpath
- **Starter Dependencies:** Pre-packaged dependency sets (spring-boot-starter-*)
- **Embedded Servers:** Tomcat, Jetty, Undertow — no external server needed
- **Actuator:** Built-in production monitoring and management
- **Spring Initializr:** Web-based project generator
- **DevTools:** Hot reload, automatic restart during development
- **Externalized Configuration:** Profiles, YAML, environment variables

Q: What does `@SpringBootApplication` do?

A: `@SpringBootApplication` is a convenience annotation that combines three annotations:

- `@SpringBootConfiguration` (extends `@Configuration`) — marks the class as a source of bean definitions
- `@EnableAutoConfiguration` — enables Spring Boot's auto-configuration mechanism
- `@ComponentScan` — enables scanning for components in the current package and sub-packages

```
@SpringBootApplication
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

Q: How does `SpringApplication.run()` work internally?

A: The startup sequence:

- 1. Creates a `SpringApplication` instance
- 2. Determines application type (Servlet, Reactive, or None)
- 3. Loads `SpringApplicationRunListeners`
- 4. Prepares the environment (properties, profiles)
- 5. Creates the `ApplicationContext`
- 6. Prepares the context (loads bean definitions)
- 7. Refreshes the context (creates beans, auto-configures)
- 8. Calls `ApplicationRunner` and `CommandLineRunner` beans
- 9. Starts the embedded server

Q: What is the difference between `application.properties` and `application.yml` ?

A: Both serve the same purpose — externalizing configuration. The differences are:

Feature	<code>application.properties</code>	<code>application.yml</code>
Format	Key=Value flat structure	YAML hierarchical structure
Readability	Can be verbose for nested properties	More readable for nested config
Multi-profile	Separate files per profile	Single file with <code>---</code> separator
Lists	<code>items[0]=a, items[1]=b</code>	<code>items:\n - a\n - b</code>

Q: What is the property resolution order in Spring Boot?

A: From highest to lowest priority:

- 1. Command line arguments
- 2. JNDI attributes
- 3. System properties (`System.getProperties()`)
- 4. OS environment variables
- 5. `application-{profile}.properties` (outside JAR)
- 6. `application-{profile}.properties` (inside JAR)
- 7. `application.properties` (outside JAR)
- 8. `application.properties` (inside JAR)
- 9. `@PropertySource` annotations
- 10. Default properties (`SpringApplication.setDefaultProperties`)

Q: What is `@ConfigurationProperties` ?

A: `@ConfigurationProperties` binds external configuration (properties/YAML) to a Java object in a type-safe way. It supports validation, nested objects, lists, and relaxed binding.

```
@ConfigurationProperties(prefix = "app.mail")
@Validated
public class MailProperties {
    @NotBlank
    private String host;
    private int port = 587;
    private List<String> recipients;
    // getters and setters
}
```

Enable with `@EnableConfigurationProperties(MailProperties.class)` or `@ConfigurationPropertiesScan`.

Q: What are `CommandLineRunner` and `ApplicationRunner` ?

A: Both interfaces allow running code after the application context is fully initialized.

`CommandLineRunner` receives arguments as `String[]`, while `ApplicationRunner` receives them as `ApplicationArguments` (parsed with options support). Use `@Order` to control execution order when multiple runners exist.

5. Spring Boot Auto-Configuration & Starters

Q: How does Spring Boot Auto-Configuration work?

A: Auto-configuration works through these steps:

- 1. `@EnableAutoConfiguration` triggers the mechanism
- 2. Spring loads `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`
- 3. Each auto-configuration class uses `@Conditional` annotations to decide whether to apply
- 4. Conditions check for class presence, bean existence, property values, etc.
- 5. User-defined beans always take precedence over auto-configured ones

Q: What are the common `@Conditional` annotations?

A:

Annotation	Condition
<code>@ConditionalOnClass</code>	Class is on classpath
<code>@ConditionalOnMissingClass</code>	Class is NOT on classpath
<code>@ConditionalOnBean</code>	Bean exists in context
<code>@ConditionalOnMissingBean</code>	Bean does NOT exist
<code>@ConditionalOnProperty</code>	Property has specific value
<code>@ConditionalOnResource</code>	Resource exists on classpath
<code>@ConditionalOnWebApplication</code>	Running as web app
<code>@ConditionalOnExpression</code>	SpEL expression is true

Q: How do you create a custom Auto-Configuration?

A: Steps to create a custom auto-configuration:

```
@AutoConfiguration
@ConditionalOnClass(MyService.class)
@EnableConfigurationProperties(MyProperties.class)
public class MyAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean
    public MyService myService(MyProperties properties) {
        return new MyService(properties.getUrl());
    }
}
```

Then register in `META-INF/spring/`

`org.springframework.boot.autoconfigure.AutoConfiguration.imports :`

```
com.example.MyAutoConfiguration
```

Q: What are Spring Boot Starters? Name the important ones.

A: Starters are curated dependency descriptors — a single dependency that pulls in everything needed for a feature. Key starters:

- `spring-boot-starter-web` — Web + MVC + Tomcat
- `spring-boot-starter-data-jpa` — JPA + Hibernate
- `spring-boot-starter-security` — Spring Security
- `spring-boot-starter-test` — JUnit, Mockito, AssertJ
- `spring-boot-starter-actuator` — Monitoring endpoints
- `spring-boot-starter-validation` — Bean validation (Hibernate Validator)
- `spring-boot-starter-cache` — Caching abstraction
- `spring-boot-starter-mail` — Email support
- `spring-boot-starter-webflux` — Reactive web
- `spring-boot-starter-amqp` — RabbitMQ

Q: How do you exclude specific Auto-Configurations?

A: You can exclude auto-configurations in three ways:

```
// 1. In @SpringBootApplication
@SpringBootApplication(exclude = {DataSourceAutoConfiguration.class})

// 2. In application.properties
spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.jdbc.DataSourceAutoCo

// 3. Using @EnableAutoConfiguration
@EnableAutoConfiguration(exclude = {SecurityAutoConfiguration.class})
```

Q: What is Spring Boot DevTools?

A: DevTools enhances the development experience with:

- **Automatic Restart:** Restarts app when classpath changes
- **LiveReload:** Refreshes browser automatically
- **Property Defaults:** Disables caching for templates
- **H2 Console:** Enables in-memory database console
- **Remote Debugging:** Support for remote development

DevTools is automatically disabled in production (when running as a packaged JAR).

6. Spring Boot Actuator & Monitoring

Q: What is Spring Boot Actuator?

A: Actuator provides production-ready features for monitoring and managing your application through HTTP endpoints or JMX. It exposes operational information about health, metrics, environment, beans, mappings, and more.

Q: What are the important Actuator endpoints?

A:

Endpoint	Description
<code>/actuator/health</code>	Application health status
<code>/actuator/info</code>	Application information
<code>/actuator/metrics</code>	Application metrics (memory, CPU, etc.)
<code>/actuator/env</code>	Environment properties
<code>/actuator/beans</code>	All registered beans
<code>/actuator/mappings</code>	All @RequestMapping paths
<code>/actuator/loggers</code>	View/modify log levels at runtime
<code>/actuator/threaddump</code>	Thread dump
<code>/actuator/httptrace</code>	Recent HTTP requests
<code>/actuator/shutdown</code>	Gracefully shut down (disabled by default)

Q: How do you create a custom Health Indicator?

A: Implement `HealthIndicator` and return a `Health` object:

```
@Component
public class DatabaseHealthIndicator implements HealthIndicator {
    @Override
    public Health health() {
        try {
            // check database connectivity
            return Health.up()
                .withDetail("database", "PostgreSQL")
                .withDetail("status", "reachable")
                .build();
        } catch (Exception e) {
            return Health.down()
                .withException(e)
                .build();
        }
    }
}
```

Q: How do you secure Actuator endpoints?

A: Configure in `application.properties`:

```
# Expose only specific endpoints
management.endpoints.web.exposure.include=health,info,metrics

# Require authentication via Spring Security
management.endpoint.health.show-details=when-authorized

# Change base path
management.endpoints.web.base-path=/manage

# Run on different port
management.server.port=9090
```

Q: How do you create custom Actuator metrics?

A: Use `MeterRegistry` from Micrometer:

```
@Service
public class OrderService {
    private final Counter orderCounter;
    private final Timer orderTimer;

    public OrderService(MeterRegistry registry) {
        this.orderCounter = registry.counter("orders.created");
        this.orderTimer = registry.timer("orders.processing.time");
    }

    public Order createOrder(OrderRequest request) {
        return orderTimer.record(() -> {
            Order order = processOrder(request);
            orderCounter.increment();
            return order;
        });
    }
}
```

7. Spring MVC & REST APIs

Q: What is Spring MVC and how does it work?

A: Spring MVC is the web module of Spring Framework based on the Model-View-Controller pattern. The request flow is:

- 1. Client sends HTTP request to `DispatcherServlet` (Front Controller)
- 2. `DispatcherServlet` consults `HandlerMapping` to find the correct controller
- 3. Controller processes request, returns `ModelAndView` or response body
- 4. `ViewResolver` resolves the view (for MVC) or `HttpMessageConverter` converts to JSON/XML (for REST)
- 5. Response sent back to client

Q: What is the difference between `@Controller` and `@RestController` ?

A: `@RestController` = `@Controller` + `@ResponseBody`. With `@Controller`, methods return view names by default (for server-side rendering). With `@RestController`, every method's return value is serialized directly to the HTTP response body (JSON/XML). Use `@Controller` for MVC (Thymeleaf/JSP) and `@RestController` for REST APIs.

Q: Explain the important annotations for REST controllers.

A:

Annotation	Purpose
<code>@RequestMapping</code>	Maps HTTP request to handler method (any HTTP method)
<code>@GetMapping</code>	Shortcut for GET requests
<code>@PostMapping</code>	Shortcut for POST requests
<code>@PutMapping</code>	Shortcut for PUT requests
<code>@DeleteMapping</code>	Shortcut for DELETE requests
<code>@PatchMapping</code>	Shortcut for PATCH requests
<code>@PathVariable</code>	Extracts value from URI path
<code>@RequestParam</code>	Extracts query parameter
<code>@RequestBody</code>	Deserializes request body to object
<code>@ResponseBody</code>	Serializes return value to response body
<code>@RequestHeader</code>	Extracts HTTP header value

Q: How do you handle exceptions globally in Spring MVC?

A: Use `@ControllerAdvice` with `@ExceptionHandler` for centralized exception handling:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse handleNotFound(ResourceNotFoundException ex) {
        return new ErrorResponse("NOT_FOUND", ex.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ErrorResponse handleValidation(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors()
            .forEach(e -> errors.put(e.getField(), e.getDefaultMessage()));
        return new ErrorResponse("VALIDATION_FAILED", errors);
    }

    @ExceptionHandler(Exception.class)
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public ErrorResponse handleGeneral(Exception ex) {
        return new ErrorResponse("INTERNAL_ERROR", "An unexpected error occurred");
    }
}
```


Q: What is `ResponseEntity` and when do you use it?

A: `ResponseEntity` represents the entire HTTP response — status code, headers, and body. Use it when you need full control over the response:

```
@GetMapping("/users/{id}")
public ResponseEntity<User> getUser(@PathVariable Long id) {
    return userService.findById(id)
        .map(user -> ResponseEntity.ok()
            .header("X-Custom-Header", "value")
            .body(user))
        .orElse(ResponseEntity.notFound().build());
}

@PostMapping("/users")
public ResponseEntity<User> createUser(@Valid @RequestBody UserDto dto) {
    User user = userService.create(dto);
    URI location = ServletUriComponentsBuilder.fromCurrentRequest()
        .path("/{id}").buildAndExpand(user.getId()).toUri();
    return ResponseEntity.created(location).body(user);
}
```

Q: How does request validation work in Spring?

A: Spring integrates with Bean Validation (JSR-380). Add `spring-boot-starter-validation` and use annotations:

```
public class UserDto {
    @NotBlank(message = "Name is required")
    private String name;

    @Email(message = "Invalid email format")
    @NotNull
    private String email;

    @Min(18) @Max(150)
    private int age;

    @Size(min = 8, message = "Password must be at least 8 characters")
    private String password;
}

@PostMapping("/users")
public User createUser(@Valid @RequestBody UserDto dto) {
    // Validation errors automatically return 400 Bad Request
}
```

Q: What are Interceptors in Spring MVC?

A: Interceptors (`HandlerInterceptor`) intercept requests before/after controller execution. They are similar to servlet filters but with access to the handler and model. Use cases: logging, authentication, locale resolution, timing.

```
@Component
public class LoggingInterceptor implements HandlerInterceptor {
    public boolean preHandle(HttpServletRequest request,
        HttpServletResponse response, Object handler) {
        log.info("Request: {} {}", request.getMethod(), request.getRequestURI());
        return true; // continue processing
    }
    public void postHandle(...) { /* after controller, before view */ }
    public void afterCompletion(...) { /* after response sent */ }
}
```

Q: What is content negotiation in Spring MVC?

A: Content negotiation determines the response format (JSON, XML, etc.) based on the client's request. Spring resolves this via: `Accept` header, URL extension (`.json` , `.xml`), or request parameter (`?format=json`). Jackson handles JSON and `jackson-dataformat-xml` handles XML. Configure with `ContentNegotiationConfigurer` .

Q: How do you implement CORS in Spring?

A: Three approaches:

```
// 1. Method-level
@CrossOrigin(origins = "http://frontend.com")
@GetMapping("/api/data")
public Data getData() { }

// 2. Controller-level
@CrossOrigin(origins = "*", maxAge = 3600)
@RestController
public class ApiController { }

// 3. Global configuration (recommended)
@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("http://frontend.com")
            .allowedMethods("GET", "POST", "PUT", "DELETE")
            .allowCredentials(true);
    }
}
```

8. Spring Data JPA & Hibernate

Q: What is Spring Data JPA?

A: Spring Data JPA is a module that simplifies data access by reducing boilerplate code. It provides repository abstractions over JPA (Java Persistence API) and Hibernate, enabling CRUD operations, custom queries, pagination, and sorting with minimal code. You define an interface, and Spring generates the implementation at runtime.

Q: What are the main Repository interfaces?

A: Hierarchy (each extends the previous):

Interface	Features
<code>Repository<T, ID></code>	Marker interface only
<code>CrudRepository</code>	Basic CRUD: save, findById, findAll, delete, count
<code>ListCrudRepository</code>	Same as Crud but returns List instead of Iterable
<code>PagingAndSortingRepository</code>	Adds pagination and sorting
<code>JpaRepository</code>	JPA-specific: flush, batch operations, example queries

```
public interface UserRepository extends JpaRepository<User, Long> {  
    List<User> findByEmail(String email);  
    List<User> findByNameContainingIgnoreCase(String name);  
    Optional<User> findByEmailAndActiveTrue(String email);  
}
```

Q: How do derived query methods work?

A: Spring Data JPA generates SQL from method names by parsing keywords:

Method Name	Generated SQL (Where clause)
<code>findByName</code>	<code>WHERE name = ?</code>
<code>findByNameAndAge</code>	<code>WHERE name = ? AND age = ?</code>
<code>findByNameOrEmail</code>	<code>WHERE name = ? OR email = ?</code>
<code>findByAgeBetween</code>	<code>WHERE age BETWEEN ? AND ?</code>
<code>findByNameLike</code>	<code>WHERE name LIKE ?</code>
<code>findByNameContaining</code>	<code>WHERE name LIKE %?%</code>
<code>findByAgeGreaterThan</code>	<code>WHERE age > ?</code>
<code>findByActiveTrue</code>	<code>WHERE active = true</code>
<code>findByNameOrderByAgeDesc</code>	<code>WHERE name = ? ORDER BY age DESC</code>
<code>countByStatus</code>	<code>SELECT count(*) WHERE status = ?</code>
<code>deleteByStatus</code>	<code>DELETE WHERE status = ?</code>
<code>existsByEmail</code>	<code>SELECT CASE WHEN count > 0 THEN true</code>

Q: What is the N+1 query problem and how do you solve it?

A: The N+1 problem occurs when fetching an entity with a lazy collection — 1 query loads the parent, then N queries load each child. Solutions:

- **JOIN FETCH in JPQL:** `@Query("SELECT u FROM User u JOIN FETCH u.orders")`
- **@EntityGraph:** Defines which associations to eagerly fetch
- **@BatchSize:** Loads lazy collections in batches
- **DTO Projections:** Fetch only needed columns

```
@EntityGraph(attributePaths = {"orders", "orders.items"})
List<User> findAllWithOrders();

// Or with @Query
@Query("SELECT DISTINCT u FROM User u LEFT JOIN FETCH u.orders o " +
      "LEFT JOIN FETCH o.items WHERE u.active = true")
List<User> findActiveUsersWithOrders();
```

Q: What is the difference between FetchType.LAZY and FetchType.EAGER?

A: `LAZY` loads the association only when accessed (default for `@OneToMany`, `@ManyToMany`). `EAGER` loads it immediately with the parent entity (default for `@ManyToOne`, `@OneToOne`). Best practice: always use `LAZY` and fetch eagerly only when needed using `JOIN FETCH` or `@EntityGraph`.

Q: How do you implement pagination and sorting?

A: Use `Pageable` and `Sort`:

```
// Repository
Page<User> findByActiveTrue(Pageable pageable);

// Service
Pageable pageable = PageRequest.of(0, 20, Sort.by("name").ascending());
Page<User> page = userRepository.findByActiveTrue(pageable);
// page.getContent(), page.getTotalElements(), page.getTotalPages()

// Controller - Spring auto-resolves Pageable from query params
@GetMapping("/users")
public Page<User> getUsers(
    @PageableDefault(size = 20, sort = "name") Pageable pageable) {
    return userRepository.findByActiveTrue(pageable);
}
// GET /users?page=0&size=20&sort=name,desc
```

Q: What are JPA Auditing annotations?

A: Spring Data JPA provides automatic auditing for entity timestamps and user tracking:

```
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public abstract class BaseEntity {

    @CreatedDate
    private LocalDateTime createdAt;

    @LastModifiedDate
    private LocalDateTime updatedAt;

    @CreatedBy
    private String createdBy;

    @LastModifiedBy
    private String updatedBy;
}

// Enable with @EnableJpaAuditing on a @Configuration class
```

Q: What is the difference between `save()` and `saveAndFlush()` ?

A: `save()` persists the entity but may delay the SQL statement until the transaction commits or a flush occurs. `saveAndFlush()` immediately sends the SQL to the database. Use `saveAndFlush()` when you need the auto-generated ID immediately or when another query depends on the saved data within the same transaction.

Q: What are Projections in Spring Data JPA?

A: Projections allow fetching partial data instead of full entities:

```
// Interface-based projection (closed)
public interface UserSummary {
    String getName();
    String getEmail();
}
List<UserSummary> findByActiveTrue();

// Class-based (DTO) projection
public record UserDto(String name, String email) {}

@Query("SELECT new com.example.UserDto(u.name, u.email) FROM User u")
List<UserDto> findAllUserDtos();

// Dynamic projection
<T> List<T> findByActiveTrue(Class<T> type);
```


9. Spring Security

Q: What is Spring Security?

A: Spring Security is a powerful authentication and authorization framework for Java applications. It provides comprehensive security services including authentication, authorization, CSRF protection, session management, password encoding, OAuth2 support, and protection against common exploits (XSS, clickjacking, etc.).

Q: How does Spring Security's Filter Chain work?

A: Spring Security uses a chain of servlet filters (`SecurityFilterChain`) that intercept every HTTP request. Key filters in order:

- `SecurityContextPersistenceFilter` — loads `SecurityContext` from session
- `CsrfFilter` — validates CSRF tokens
- `LogoutFilter` — processes logout requests
- `UsernamePasswordAuthenticationFilter` — processes form login
- `BasicAuthenticationFilter` — processes HTTP Basic auth
- `BearerTokenAuthenticationFilter` — processes JWT/OAuth2 tokens
- `ExceptionTranslationFilter` — handles security exceptions
- `FilterSecurityInterceptor / AuthorizationFilter` — final authorization check

Q: How do you configure Spring Security with the modern approach?

A: Since Spring Security 5.7+, use `SecurityFilterChain` bean (the `WebSecurityConfigurerAdapter` is deprecated):

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/public/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .requestMatchers("/api/**").authenticated()
                .anyRequest().permitAll()
            )
            .httpBasic(Customizer.withDefaults())
            .build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

Q: What is the difference between Authentication and Authorization?

A:

Authentication	Authorization
Verifies WHO you are	Determines WHAT you can access
Login process	Permission checking
401 Unauthorized if fails	403 Forbidden if fails
Handled by AuthenticationManager	Handled by AccessDecisionManager
Example: username/password check	Example: role-based access control

Q: How do you implement JWT-based authentication in Spring Boot?

A: The flow involves: a custom filter that extracts and validates the JWT from the Authorization header, sets the SecurityContext, and integrates with UserDetailsService:

```
@Component
public class JwtAuthFilter extends OncePerRequestFilter {
    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response, FilterChain chain)
        throws ServletException, IOException {
        String header = request.getHeader("Authorization");
        if (header == null || !header.startsWith("Bearer ")) {
            chain.doFilter(request, response);
            return;
        }
        String token = header.substring(7);
        String username = jwtService.extractUsername(token);

        if (username != null &&
            SecurityContextHolder.getContext().getAuthentication() == null) {
            UserDetails user = userDetailsService.loadUserByUsername(username);
            if (jwtService.isTokenValid(token, user)) {
                UsernamePasswordAuthenticationToken auth =
                    new UsernamePasswordAuthenticationToken(
                        user, null, user.getAuthorities());
                SecurityContextHolder.getContext().setAuthentication(auth);
            }
        }
        chain.doFilter(request, response);
    }
}
```

Q: What is method-level security?

A: Spring Security supports authorization at method level using annotations. Enable with `@EnableMethodSecurity`:

```
@EnableMethodSecurity // on @Configuration class

@Service
public class UserService {
    @PreAuthorize("hasRole('ADMIN')")
    public void deleteUser(Long id) { }

    @PreAuthorize("#userId == authentication.principal.id or hasRole('ADMIN')")
    public User getUser(@Param("userId") Long userId) { }

    @PostAuthorize("returnObject.owner == authentication.name")
    public Document getDocument(Long docId) { }

    @PreFilter("filterObject.owner == authentication.name")
    public void processDocuments(List<Document> docs) { }
}
```

Q: What is CSRF and how does Spring Security handle it?

A: CSRF (Cross-Site Request Forgery) is an attack where a malicious site tricks the browser into making requests to your application using the victim's session. Spring Security includes CSRF protection by default — it generates a token that must be included in state-changing requests (POST, PUT, DELETE). For stateless REST APIs using JWT, CSRF protection is typically disabled since there are no cookies/sessions to exploit.

Q: What is OAuth2 and how does Spring Security support it?

A: OAuth2 is an authorization framework that allows third-party applications to obtain limited access to a user's resources. Spring Security supports:

- **OAuth2 Login:** Login with Google, GitHub, etc. (`spring-boot-starter-oauth2-client`)
- **OAuth2 Resource Server:** Validate JWT tokens from an authorization server (`spring-boot-starter-oauth2-resource-server`)
- **Authorization Server:** Spring Authorization Server project for building your own

```
# application.yml for Google OAuth2 Login
spring:
  security:
    oauth2:
      client:
        registration:
          google:
            client-id: ${GOOGLE_CLIENT_ID}
            client-secret: ${GOOGLE_CLIENT_SECRET}
            scope: openid, profile, email
```

10. Spring AOP (Aspect-Oriented Programming)

Q: What is AOP and why is it used?

A: AOP (Aspect-Oriented Programming) enables modularization of cross-cutting concerns — functionality that spans multiple modules like logging, security, transactions, and caching. Instead of scattering this code across every method, you define it once in an **Aspect** and declare where it applies. Spring AOP uses **proxy-based** mechanism (JDK dynamic proxies or CGLIB).

Q: What are the key AOP terminology/concepts?

A:

Term	Definition
Aspect	A module encapsulating cross-cutting concern (<code>@Aspect</code> class)
Join Point	A point during execution (method call, exception throw)
Advice	Action taken at a join point (Before, After, Around)
Pointcut	Expression that matches join points
Target Object	The object being advised (proxied)
Weaving	Linking aspects with application objects (compile-time, load-time, or runtime)

Q: What are the types of Advice?

A:

- `@Before` — runs before the method executes
- `@After` — runs after the method (regardless of outcome)
- `@AfterReturning` — runs after successful return
- `@AfterThrowing` — runs after an exception is thrown
- `@Around` — wraps the method, controls if/when it executes (most powerful)

```
@Aspect
@Component
public class LoggingAspect {

    @Around("execution(* com.example.service.*(..))")
    public Object logExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = joinPoint.proceed();
        long duration = System.currentTimeMillis() - start;
        log.info("{} executed in {} ms",
            joinPoint.getSignature(), duration);
        return result;
    }

    @AfterThrowing(pointcut = "execution(* com.example.*(..))",
        throwing = "ex")
    public void logException(JoinPoint joinPoint, Exception ex) {
        log.error("Exception in {}: {}",
            joinPoint.getSignature(), ex.getMessage());
    }
}
```

Q: What are common Pointcut expressions?

A:

```
// Match all methods in service package
execution(* com.example.service.*.*(..))

// Match specific method with return type
execution(String com.example.service.UserService.findByEmail(String))

// Match all public methods
execution(public * *(..))

// Match methods annotated with @Transactional
@annotation(org.springframework.transaction.annotation.Transactional)

// Match beans named *Service
bean(*Service)

// Within a specific class
within(com.example.service.OrderService)

// Combine with logical operators
execution(* com.example.service.*.*(..)) && !execution(* *.get*(..))
```

Q: What is the limitation of Spring AOP?

A: Spring AOP only supports **method-level join points** (not field access or constructor calls). It uses proxies, so: (1) aspects only work on Spring-managed beans, (2) self-invocation (calling another method within the same class) bypasses the proxy and the aspect won't trigger, (3) only public methods can be advised. For full AOP support (compile-time or load-time weaving, field interception), use AspectJ directly.

11. Spring Transaction Management

Q: How does `@Transactional` work in Spring?

A: Spring creates a proxy around the bean. When a `@Transactional` method is called, the proxy: (1) begins a transaction, (2) executes the method, (3) commits on success or rolls back on exception. By default, it rolls back on unchecked exceptions (`RuntimeException`) and commits on checked exceptions.

```
@Service
@Transactional(readonly = true) // default for all methods
public class OrderService {

    @Transactional // override: read-write
    public Order placeOrder(OrderRequest request) {
        Order order = orderRepository.save(new Order(request));
        inventoryService.deductStock(request.getItems());
        paymentService.charge(request.getPayment());
        return order;
    }

    public List<Order> getOrders(Long userId) {
        return orderRepository.findByUserId(userId); // read-only
    }
}
```

Q: What are the Transaction Propagation types?

A:

Propagation	Behavior
<code>REQUIRED</code> (default)	Join existing or create new transaction
<code>REQUIRES_NEW</code>	Always create new; suspend existing
<code>SUPPORTS</code>	Join if exists; run without if not
<code>NOT_SUPPORTED</code>	Suspend existing; run non-transactionally
<code>MANDATORY</code>	Must join existing; throw if none
<code>NEVER</code>	Throw if transaction exists
<code>NESTED</code>	Nested transaction with savepoint

Q: What are the Transaction Isolation levels?

A:

Level	Dirty Read	Non-Repeatable Read	Phantom Read
<code>READ_UNCOMMITTED</code>	Yes	Yes	Yes
<code>READ_COMMITTED</code>	No	Yes	Yes
<code>REPEATABLE_READ</code>	No	No	Yes
<code>SERIALIZABLE</code>	No	No	No

Q: Why doesn't `@Transactional` work on self-invocation?

A: Since Spring uses proxies for `@Transactional`, calling a transactional method from within the same class bypasses the proxy — the call goes directly to the target object.

Solutions:

- Extract the method to a separate service class
- Inject `self` using `@Lazy` or `ObjectProvider`
- Use `TransactionTemplate` for programmatic transactions

Q: What is `@Transactional(readOnly = true)` ?

A: It hints to the JPA provider and database that no data modification will occur. Benefits: Hibernate skips dirty checking (performance gain), the database can optimize queries, and it clearly communicates intent. Always use for read operations.

12. Spring Cloud & Microservices

Q: What is Spring Cloud?

A: Spring Cloud provides tools for building distributed systems and microservices patterns: service discovery, configuration management, circuit breakers, API gateway, distributed tracing, and load balancing. It builds on Spring Boot and integrates with Netflix OSS, Consul, Kubernetes, and other infrastructure tools.

Q: What are the main Spring Cloud components?

A:

Component	Purpose	Implementation
Service Discovery	Register and find services	Eureka, Consul, Zookeeper
API Gateway	Single entry point, routing	Spring Cloud Gateway
Config Server	Centralized configuration	Spring Cloud Config
Circuit Breaker	Fault tolerance	Resilience4j
Load Balancer	Client-side load balancing	Spring Cloud LoadBalancer
Distributed Tracing	Request tracing across services	Micrometer Tracing + Zipkin
Stream	Event-driven messaging	Spring Cloud Stream

Q: What is Service Discovery and how does Eureka work?

A: Service Discovery allows microservices to find each other without hardcoded URLs. Eureka consists of a **Server** (registry) and **Clients** (services). Each service registers itself with the Eureka server on startup and sends heartbeats. Other services query Eureka to find available instances.

```
// Eureka Server
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApp { }

// Eureka Client (in each microservice)
// application.yml
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
  instance:
    prefer-ip-address: true
```

Q: What is Spring Cloud Gateway?

A: Spring Cloud Gateway is a reactive API gateway that provides routing, filtering, rate limiting, and circuit breaking. It replaces Zuul (now in maintenance mode).

```
spring:
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: lb://USER-SERVICE
          predicates:
            - Path=/api/users/**
          filters:
            - StripPrefix=1
            - name: CircuitBreaker
              args:
                name: userCircuitBreaker
                fallbackUri: forward:/fallback/users
        - id: order-service
          uri: lb://ORDER-SERVICE
          predicates:
            - Path=/api/orders/**
            - Method=GET,POST
```

Q: What is a Circuit Breaker pattern? How does Resilience4j work?

A: Circuit Breaker prevents cascading failures when a downstream service is unavailable. States: **CLOSED** (normal), **OPEN** (requests fail fast), **HALF_OPEN** (test with limited requests).

```
@Service
public class OrderService {

    @CircuitBreaker(name = "inventoryService",
                    fallbackMethod = "inventoryFallback")
    @Retry(name = "inventoryService")
    @TimeLimiter(name = "inventoryService")
    public CompletableFuture<Boolean> checkInventory(String productId) {
        return CompletableFuture.supplyAsync(
            () -> inventoryClient.isInStock(productId));
    }

    private CompletableFuture<Boolean> inventoryFallback(
        String productId, Throwable t) {
        log.warn("Fallback for inventory check: {}", t.getMessage());
        return CompletableFuture.completedFuture(false);
    }
}
```

Q: What is Spring Cloud Config?

A: Spring Cloud Config provides centralized configuration management for microservices. A Config Server serves configuration from Git, filesystem, or Vault. Clients fetch configuration at startup and can be refreshed at runtime via `/actuator/refresh` or Spring Cloud Bus for broadcasting changes to all instances.

Q: How do microservices communicate?

A: Two main patterns:

- **Synchronous:** REST (RestTemplate/WebClient), gRPC, OpenFeign client
- **Asynchronous:** Message queues (RabbitMQ, Kafka), event-driven via Spring Cloud Stream

```
// OpenFeign Client (declarative REST client)
@FeignClient(name = "inventory-service")
public interface InventoryClient {
    @GetMapping("/api/inventory/{productId}")
    InventoryResponse checkStock(@PathVariable String productId);
}
```

Q: What is the Saga Pattern?

A: The Saga pattern manages distributed transactions across microservices. Instead of ACID transactions, each service performs its local transaction and publishes an event. If a step fails, compensating transactions are executed to undo previous steps. Two approaches: **Choreography** (event-driven, decentralized) and **Orchestration** (central coordinator manages the saga).

13. Spring Batch

Q: What is Spring Batch?

A: Spring Batch is a framework for batch processing — executing large-volume, non-interactive jobs like data migration, ETL processes, report generation, and file processing. It provides reusable components for reading, processing, and writing data with support for transactions, restartability, skip/retry, and job scheduling.

Q: What are the core components of Spring Batch?

A:

Component	Role
<code>Job</code>	The entire batch process (contains Steps)
<code>Step</code>	An independent phase of a Job
<code>ItemReader</code>	Reads data from a source (DB, file, queue)
<code>ItemProcessor</code>	Transforms/validates data
<code>ItemWriter</code>	Writes processed data to target
<code>JobRepository</code>	Stores metadata about job executions
<code>JobLauncher</code>	Starts/triggers a Job

Q: How do you configure a chunk-oriented Step?

A:

```
@Bean
public Step importStep(JobRepository jobRepository,
    PlatformTransactionManager txManager) {
    return new StepBuilder("importStep", jobRepository)
        .<InputRecord, OutputRecord>chunk(100, txManager)
        .reader(csvFileReader())
        .processor(validationProcessor())
        .writer(databaseWriter())
        .faultTolerant()
        .skipLimit(10)
        .skip(ParseException.class)
        .retryLimit(3)
        .retry(DeadlockLoserDataAccessException.class)
        .build();
}

@Bean
public Job importJob(JobRepository jobRepository, Step importStep) {
    return new JobBuilder("importJob", jobRepository)
        .start(importStep)
        .build();
}
```

14. Spring WebFlux & Reactive Programming

Q: What is Spring WebFlux?

A: Spring WebFlux is the reactive-stack web framework in Spring, designed for non-blocking, asynchronous applications. It runs on Netty (default), Undertow, or Servlet 3.1+ containers. It uses **Project Reactor** with `Mono` (0 or 1 element) and `Flux` (0 to N elements) as reactive types.

Q: What is the difference between Spring MVC and Spring WebFlux?

A:

Feature	Spring MVC	Spring WebFlux
Programming Model	Imperative (blocking)	Reactive (non-blocking)
Thread Model	Thread-per-request	Event loop (few threads)
Server	Tomcat, Jetty	Netty, Undertow
Return Types	POJO, ResponseEntity	Mono, Flux
Use Case	Traditional CRUD apps	High-concurrency, streaming, IoT
Data Access	JDBC, JPA	R2DBC, reactive MongoDB

Q: Show a basic WebFlux controller example.

A:

```
@RestController
@RequestMapping("/api/users")
public class UserController {
    private final UserRepository userRepository; // ReactiveCrudRepository

    @GetMapping
    public Flux<User> getAllUsers() {
        return userRepository.findAll();
    }

    @GetMapping("/{id}")
    public Mono<ResponseEntity<User>> getUser(@PathVariable String id) {
        return userRepository.findById(id)
            .map(ResponseEntity::ok)
            .defaultIfEmpty(ResponseEntity.notFound().build());
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Mono<User> createUser(@Valid @RequestBody User user) {
        return userRepository.save(user);
    }
}
```

Q: What is `WebClient` and how does it differ from `RestTemplate` ?

A: `WebClient` is the modern, non-blocking HTTP client in Spring 5+ (replacement for `RestTemplate` , which is now in maintenance mode):

```
WebClient client = WebClient.builder()
    .baseUrl("http://api.example.com")
    .defaultHeader("Authorization", "Bearer " + token)
    .build();

// Non-blocking call
Mono<User> user = client.get()
    .uri("/users/{id}", userId)
    .retrieve()
    .bodyToMono(User.class);

// Blocking (if needed in MVC context)
User user = client.get()
    .uri("/users/{id}", userId)
    .retrieve()
    .bodyToMono(User.class)
    .block();
```

15. Spring Testing

Q: What testing support does Spring Boot provide?

A: `spring-boot-starter-test` includes:

- **JUnit 5** — test framework
- **Mockito** — mocking framework
- **AssertJ** — fluent assertions
- **Spring Test** — integration test support
- **JSONassert** — JSON assertion
- **Testcontainers** — real database containers for testing

Q: What is the difference between `@SpringBootTest` and slice tests?

A: `@SpringBootTest` loads the full application context — good for integration tests but slow. Slice tests load only relevant parts:

Annotation	What It Loads	Use Case
<code>@WebMvcTest</code>	Controllers, filters, converters	Controller unit tests
<code>@DataJpaTest</code>	JPA repositories, entities	Repository tests
<code>@WebFluxTest</code>	WebFlux controllers	Reactive controller tests
<code>@JsonTest</code>	JSON serialization/deserialization	DTO mapping tests
<code>@RestClientTest</code>	REST client components	External API call tests

Q: How do you write a controller test with `@WebMvcTest` ?

A:

```
@WebMvcTest(UserController.class)
class UserControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockitoBean
    private UserService userService;

    @Test
    void shouldReturnUser() throws Exception {
        User user = new User(1L, "John", "john@email.com");
        when(userService.findById(1L)).thenReturn(Optional.of(user));

        mockMvc.perform(get("/api/users/1")
            .contentType(MediaType.APPLICATION_JSON))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.name").value("John"))
            .andExpect(jsonPath("$.email").value("john@email.com"));
    }

    @Test
    void shouldReturn404WhenNotFound() throws Exception {
        when(userService.findById(99L)).thenReturn(Optional.empty());

        mockMvc.perform(get("/api/users/99"))
            .andExpect(status().isNotFound());
    }
}
```

Q: How do you test repositories with `@DataJpaTest` ?

A:

```
@DataJpaTest
class UserRepositoryTest {

    @Autowired
    private TestEntityManager entityManager;

    @Autowired
    private UserRepository userRepository;

    @Test
    void shouldFindByEmail() {
        User user = new User("John", "john@email.com");
        entityManager.persistAndFlush(user);

        Optional<User> found = userRepository.findByEmail("john@email.com");

        assertThat(found).isPresent();
        assertThat(found.get().getName()).isEqualTo("John");
    }
}
```

`@DataJpaTest` configures an in-memory H2 database, scans for `@Entity` classes, and rolls back transactions after each test.

Q: What is the role of `@MockBean` vs `@Mock` ?

A: `@MockitoBean` (Spring Boot 3.4+, replaces `@MockBean`) creates a mock and registers it in the Spring context, replacing any existing bean of the same type. `@Mock` (plain Mockito) creates a mock outside of Spring context — used in unit tests with `@ExtendWith(MockitoExtension.class)`. Use `@MockitoBean` for slice/integration tests and `@Mock` for pure unit tests.

Q: How do you use Testcontainers with Spring Boot?

A: Testcontainers spins up real databases in Docker for integration tests:

```
@SpringBootTest
@Testcontainers
class OrderIntegrationTest {

    @Container
    @ServiceConnection
    static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:16-alpine");

    @Autowired
    private OrderRepository orderRepository;

    @Test
    void shouldSaveAndRetrieveOrder() {
        Order order = new Order("ORD-001", BigDecimal.valueOf(99.99));
        orderRepository.save(order);

        Optional<Order> found = orderRepository.findByOrderNumber("ORD-001");
        assertThat(found).isPresent();
        assertThat(found.get().getAmount()).isEqualByComparingTo("99.99");
    }
}
```


16. Spring Profiles & Configuration

Q: What are Spring Profiles?

A: Profiles allow you to define environment-specific configurations (dev, staging, prod) and activate them at runtime. Beans, properties, and configurations can be tied to specific profiles.

```
# Activate profile
spring.profiles.active=dev

# Or via command line
java -jar app.jar --spring.profiles.active=prod

# Or via environment variable
SPRING_PROFILES_ACTIVE=prod
```

Q: How do you define profile-specific beans?

A:

```
@Configuration
public class DataSourceConfig {

    @Bean
    @Profile("dev")
    public DataSource devDataSource() {
        // H2 in-memory database
        return new EmbeddedDatabaseBuilder()
            .setType(EmbeddedDatabaseType.H2).build();
    }

    @Bean
    @Profile("prod")
    public DataSource prodDataSource() {
        // Production PostgreSQL
        HikariDataSource ds = new HikariDataSource();
        ds.setJdbcUrl("jdbc:postgresql://prod-db:5432/app");
        return ds;
    }
}
```

Q: What is `@Value` annotation?

A: `@Value` injects values from properties, environment variables, or SpEL expressions:

```
@Value("${app.name}")
private String appName;

@Value("${app.timeout:30}") // default value
private int timeout;

@Value("${APP_SECRET}") // environment variable
private String secret;

@Value("#{T(java.lang.Math).random() * 100}") // SpEL
private double random;

@Value("${app.servers}") // comma-separated → List
private List<String> servers;
```

17. Spring Caching

Q: How does Spring Caching work?

A: Spring provides a caching abstraction with annotations. Enable with `@EnableCaching`. Supports multiple providers: `ConcurrentHashMap` (default), `EhCache`, `Redis`, `Caffeine`, `Hazelcast`.

```
@Service
public class ProductService {

    @Cacheable(value = "products", key = "#id")
    public Product findById(Long id) {
        return productRepository.findById(id).orElseThrow();
    }

    @CachePut(value = "products", key = "#product.id")
    public Product update(Product product) {
        return productRepository.save(product);
    }

    @CacheEvict(value = "products", key = "#id")
    public void delete(Long id) {
        productRepository.deleteById(id);
    }

    @CacheEvict(value = "products", allEntries = true)
    public void clearCache() { }
}
```

Q: What is the difference between `@Cacheable`, `@CachePut`, and `@CacheEvict` ?

A:

Annotation	Behavior
<code>@Cacheable</code>	Returns cached result if exists; otherwise executes method and caches result
<code>@CachePut</code>	Always executes method and updates the cache
<code>@CacheEvict</code>	Removes entry from cache
<code>@Caching</code>	Groups multiple cache annotations

18. Spring Messaging (JMS, Kafka, RabbitMQ)

Q: How does Spring integrate with Apache Kafka?

A: Spring Kafka provides `KafkaTemplate` for producing and `@KafkaListener` for consuming:

```
// Producer
@Service
public class OrderEventProducer {
    private final KafkaTemplate<String, OrderEvent> kafkaTemplate;

    public void publishOrderCreated(OrderEvent event) {
        kafkaTemplate.send("order-events", event.getOrderId(), event);
    }
}

// Consumer
@Service
public class OrderEventConsumer {

    @KafkaListener(topics = "order-events", groupId = "inventory-service")
    public void handleOrderEvent(OrderEvent event) {
        log.info("Received order event: {}", event.getOrderId());
        inventoryService.reserveStock(event);
    }
}
```

Q: How does Spring integrate with RabbitMQ?

A: Spring AMQP provides `RabbitTemplate` for sending and `@RabbitListener` for receiving:

```
// Producer
@Service
public class NotificationProducer {
    private final RabbitTemplate rabbitTemplate;

    public void sendNotification(Notification notification) {
        rabbitTemplate.convertAndSend("notifications-exchange",
            "notification.email", notification);
    }
}

// Consumer
@Service
public class NotificationConsumer {

    @RabbitListener(queues = "email-notifications")
    public void handleNotification(Notification notification) {
        emailService.send(notification);
    }
}
```

19. Spring Design Patterns

Q: What design patterns are used in Spring Framework?

A:

Pattern	Where Used
Singleton	Default bean scope
Factory	BeanFactory, FactoryBean
Proxy	AOP, @Transactional, @Cacheable
Template Method	JdbcTemplate, RestTemplate, JmsTemplate
Observer	ApplicationEvent, @EventListener
Front Controller	DispatcherServlet
MVC	Spring MVC (Model-View-Controller)
Strategy	Resource loading, ViewResolver
Adapter	HandlerAdapter
Decorator	BeanPostProcessor
Builder	UriComponentsBuilder, ResponseEntity.ok()

Q: How do Spring Events work?

A: Spring provides a publish-subscribe event mechanism for decoupled communication within an application:

```
// Custom Event
public class OrderCreatedEvent {
    private final Order order;
    public OrderCreatedEvent(Order order) { this.order = order; }
    public Order getOrder() { return order; }
}

// Publisher
@Service
public class OrderService {
    private final ApplicationEventPublisher eventPublisher;

    @Transactional
    public Order createOrder(OrderRequest request) {
        Order order = orderRepository.save(new Order(request));
        eventPublisher.publishEvent(new OrderCreatedEvent(order));
        return order;
    }
}

// Listener
@Component
public class InventoryListener {
    @EventListener
    public void onOrderCreated(OrderCreatedEvent event) {
        inventoryService.deductStock(event.getOrder());
    }

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void afterOrderCommitted(OrderCreatedEvent event) {
        notificationService.sendConfirmation(event.getOrder());
    }
}
```

20. Scenario-Based & Advanced Questions

Q: How do you handle a slow-performing Spring Boot application?

A: Systematic approach:

- **1. Monitor:** Use Actuator metrics, Micrometer + Prometheus/Grafana
- **2. Database:** Check N+1 queries (enable `spring.jpa.show-sql`), add indexes, use pagination, optimize fetch strategies
- **3. Caching:** Add `@Cacheable` for frequently read, rarely changing data
- **4. Connection Pools:** Tune HikariCP settings (pool size, timeout)
- **5. Async:** Use `@Async` for non-critical operations
- **6. Startup:** Use lazy initialization (`spring.main.lazy-initialization=true`)
- **7. JVM:** Profile with JProfiler/YourKit, tune GC settings
- **8. Thread dumps:** Use `/actuator/heapdump` to find bottlenecks

Q: How would you design a microservices architecture with Spring Boot?

A: Key components and decisions:

- **API Gateway:** Spring Cloud Gateway for routing, rate limiting, auth
- **Service Discovery:** Eureka or Consul for dynamic service registration
- **Config Management:** Spring Cloud Config Server with Git backend
- **Communication:** REST (synchronous) + Kafka/RabbitMQ (asynchronous)
- **Resilience:** Resilience4j for circuit breakers, retries, rate limiters
- **Database:** Database-per-service pattern, avoid shared databases
- **Transactions:** Saga pattern for distributed transactions
- **Observability:** Distributed tracing (Micrometer + Zipkin), centralized logging (ELK)
- **Security:** OAuth2 + JWT, token validation at gateway
- **Deployment:** Docker containers on Kubernetes

Q: How do you handle circular dependencies in Spring?

A: Circular dependencies occur when Bean A depends on Bean B and vice versa. Spring throws `BeanCurrentlyInCreationException` with constructor injection. Solutions:

- **Best:** Refactor the design — extract common logic to a third class
- **Use `@Lazy` :** Lazy-initialize one dependency to break the cycle
- **Use Setter Injection:** Allows Spring to create both beans first, then inject
- **Use `@PostConstruct` :** Set the dependency after construction
- **Events:** Decouple with `ApplicationEvent` instead of direct calls

Q: What is the difference between `@Bean` and `@Component` ?

A:

Aspect	@Bean	@Component
Used on	Methods (inside @Configuration)	Classes
Control	Full control over instantiation logic	Auto-detected via component scanning
Third-party	Can register third-party classes as beans	Only for your own classes
Name	Method name is bean name	Class name (lowercase first letter)

Q: How do you implement async processing in Spring?

A: Enable with `@EnableAsync` and annotate methods with `@Async`:

```
@Configuration
@EnableAsync
public class AsyncConfig {
    @Bean
    public Executor taskExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(5);
        executor.setMaxPoolSize(10);
        executor.setQueueCapacity(25);
        executor.setThreadNamePrefix("async-");
        executor.initialize();
        return executor;
    }
}

@Service
public class EmailService {
    @Async
    public CompletableFuture<Boolean> sendEmail(String to, String body) {
        // runs in separate thread
        boolean sent = mailSender.send(to, body);
        return CompletableFuture.completedFuture(sent);
    }
}
```

Important: `@Async` does NOT work on self-invocation (same class method calls) due to proxy-based mechanism — same limitation as `@Transactional`.

Q: What is Spring Boot's Graceful Shutdown?

A: Graceful shutdown allows in-flight requests to complete before the application stops. Enable with:

```
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=30s
```

When shutdown is triggered: (1) server stops accepting new requests, (2) existing requests complete within the timeout, (3) `@PreDestroy` methods run, (4) application context closes.

Q: How do you implement rate limiting in Spring Boot?

A: Several approaches:

- **Spring Cloud Gateway:** Built-in `RequestRateLimiter` filter with Redis
- **Resilience4j:** `@RateLimiter` annotation
- **Bucket4j:** Token-bucket algorithm library
- **Custom Interceptor:** With Redis/in-memory counter

```
@RestController
public class ApiController {
    @GetMapping("/api/data")
    @RateLimiter(name = "default", fallbackMethod = "rateLimitFallback")
    public ResponseEntity<Data> getData() {
        return ResponseEntity.ok(service.getData());
    }

    private ResponseEntity<Data> rateLimitFallback(Throwable t) {
        return ResponseEntity.status(HttpStatus.TOO_MANY_REQUESTS).build();
    }
}
```

Q: What is the difference between `RestTemplate` and `WebClient`?

A:

Feature	RestTemplate	WebClient
Blocking	Yes (synchronous)	No (reactive, non-blocking)
Status	Maintenance mode	Actively developed
Thread Model	One thread blocked per call	Event-loop, efficient
Streaming	Not supported	Supported (Flux)
Spring Boot 3+	Use RestClient instead	Preferred for reactive

In Spring Boot 3.2+, `RestClient` is the modern synchronous HTTP client replacing `RestTemplate`.

Q: How do you schedule tasks in Spring?

A: Enable with `@EnableScheduling`:

```
@Component
public class ScheduledTasks {

    @Scheduled(fixedRate = 5000) // every 5 seconds
    public void reportStatus() { }

    @Scheduled(fixedDelay = 10000) // 10s after last completion
    public void processQueue() { }

    @Scheduled(cron = "0 0 2 * * ?") // daily at 2 AM
    public void dailyCleanup() { }

    @Scheduled(cron = "0 */15 * * * ?") // every 15 minutes
    public void syncData() { }
}
```

Q: How do you deploy a Spring Boot application?

A: Common deployment strategies:

- **Executable JAR:** `java -jar app.jar` (most common)
- **Docker:** Multi-stage Dockerfile with layered JARs
- **Kubernetes:** Container orchestration with health checks from Actuator
- **Cloud Native Buildpacks:** `./mvnw spring-boot:build-image`
- **Traditional WAR:** Deploy to external Tomcat (extend `SpringBootServletInitializer`)
- **GraalVM Native:** Compile to native binary for instant startup

```
# Multi-stage Dockerfile
FROM eclipse-temurin:21-jre-alpine
COPY target/app.jar /app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

Q: What is Spring Native / GraalVM support?

A: Spring Boot 3.x supports ahead-of-time (AOT) compilation to native executables using GraalVM. Benefits: near-instant startup (50ms vs 2-5s), lower memory usage, ideal for serverless/containers. Trade-offs: longer build time, no runtime reflection without hints, limited dynamic features. Build with: `./mvnw -Pnative native:compile`.

Q: What are the best practices for building Spring Boot applications?

A:

- **Use Constructor Injection** over field injection
- **Externalize Configuration** — never hardcode values
- **Use Profiles** for environment-specific settings
- **Enable Actuator** for production monitoring
- **Use DTOs** — never expose entities directly in APIs
- **Validate Input** at controller level with `@Valid`
- **Handle Exceptions Globally** with `@ControllerAdvice`
- **Use `@Transactional(readOnly=true)`** for read operations
- **Write Tests** — unit, integration, and slice tests
- **Use Flyway/Liquibase** for database migrations
- **Log Appropriately** — structured logging, appropriate levels
- **Secure Endpoints** — Spring Security from day one
- **Use Layered Architecture** — Controller → Service → Repository
- **API Versioning** — plan from the start (/api/v1/...)

Q: What is Spring Modulith?

A: Spring Modulith is a framework for building well-structured, modular monoliths. It enforces module boundaries within a single application, supports inter-module events, generates documentation, and makes it easier to evolve to microservices later. It provides: module structure verification, application events for decoupled communication, and integration testing support per module.

Q: Explain the CQRS pattern with Spring.

A: CQRS (Command Query Responsibility Segregation) separates read and write models:

- **Command Side:** Handles writes (create, update, delete) — uses JPA entities, `@Transactional`
- **Query Side:** Handles reads — uses optimized projections, read replicas, denormalized views
- **Event Sourcing:** Changes stored as events, state rebuilt from event stream
- **Implementation:** Separate services and repositories for commands vs queries, connected via events (Kafka/RabbitMQ)

Best of Luck!

Prepare well, stay confident, and ace your interview.



Spring Framework & Spring Boot Interview Guide — 2026 Edition